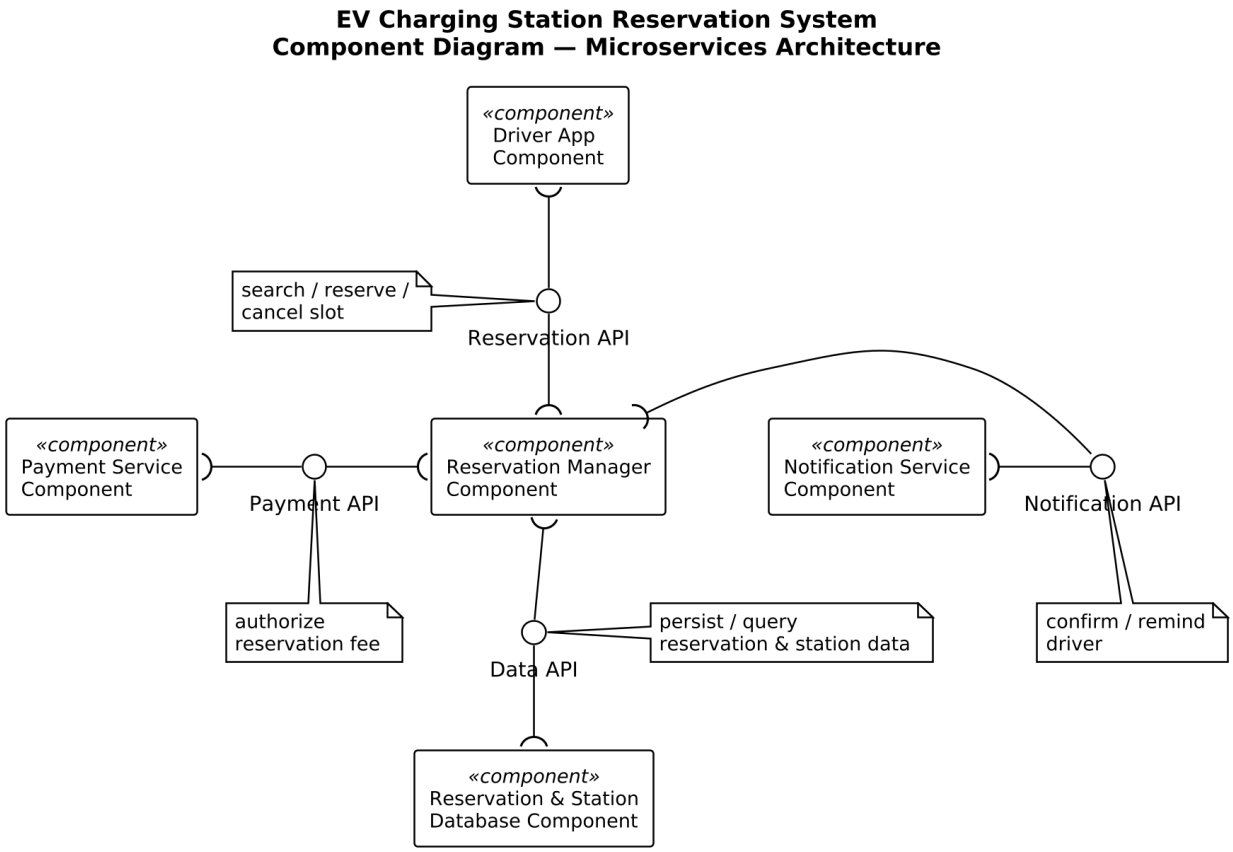
Lab 3 – Component Diagram

Anushka – PES1UG24CS074 – Section B

EV Charging Station Reservation System — Microservices Architecture



Lab 3 Submission

Anushka – PES1UG24CS074 – Section B

Architectural Justification and Component Modelling: EV Charging Station Reservation System

Architecture Selection

We chose Microservices Architecture for the EV Charging Station Reservation System. The system is decomposed into independently deployable services, a Driver App Component, a Reservation Manager Component, a Payment Service Component, a Notification Service Component, and a Reservation & Station Database Component that communicate through well-defined APIs rather than sharing a single codebase or database.

Two Reasons

1. Independent scaling under peak load: NFR-002 requires the platform to support at least 500 concurrent reservation requests with sub-2-second response times during peak commute hours. Because the Reservation Manager Component is a separate service from the Payment Service, Notification Service, and Database components, it can be horizontally scaled on its own during demand spikes without redeploying or over-provisioning the entire system, which a monolithic Layered or centralized Client-Server design would require.

2. Fault isolation across real-time data streams: FR-005 requires station-status updates (Available / Occupied / Faulty / Offline) to propagate to search results within 5 seconds, while reservation, payment, and notification flows run concurrently. Microservices let the station-status stream, the payment integration, and the notification dispatch fail or degrade independently for example, a slow third-party SMS gateway cannot stall an in-progress reservation or block another driver's station search.

Security Advantage

Isolating the Payment Service Component as its own microservice means the Driver App and Reservation Manager never handle raw card data directly, the Reservation Manager only sends an authorization request over the Payment API and receives an approve/decline response. Sensitive payment-data handling stays confined to a single, independently hardened and audited service (supporting the secure payment-data handling requirement), which shrinks the attack surface compared with a design where payment logic is embedded in a shared application tier.

Performance Benefit

Because reservations must complete in under 60 seconds end-to-end, the Reservation Manager Component can call the Notification Service asynchronously (fire-and-forget) via the Notification API after confirming a slot, instead of waiting for the confirmation message to be delivered before returning a response to the driver. Decoupling the notification and database-persistence services from the critical reservation path keeps the driver-facing response fast even if the Notification Service is briefly under load, directly satisfying the scenario's under-60-second and sub-2-second response-time constraints.